

# Advanced Data Management

## Query answering in ontologies

**Domenico Fabio Savo**

*Corso di laurea magistrale  
INGEGNERIA INFORMATICA  
Dipartimento di Ingegneria Gestionale, dell'Informazione e della Produzione  
University of Bergamo*

---

# Outline of the course

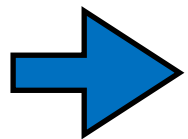
1. Introduction to propositional logic
  2. Introduction to First-order Logic
  3. Relational Calculus
  4. Information Integration Systems (IIS)
  5. Logical formalization of IISs
  6. Mapping between **Global Schema** e **Data Sources**
  7. Incomplete information databases
  8. Query answering over IISs
  9. Ontology-Based Data Management (OBDM)
  10. Description Logic Ontologies
  11. **Query answering in ontologies**
  12. Query answering in OBDM
-

# Query answering in ontologies

---

# Query answering in tradition databases

- Data are completely specified (**Close Word Assumption**), and are typically large.
- Intensional information (the schema) is used **only** in the design phase. At **runtime**, the data is assumed to satisfy the schema, and therefore the **schema is not used**.
- Queries allow for complex navigation paths in the data (cf. SQL).

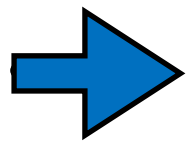


Query answering amounts to **query evaluation**, which is computationally easy.

# Query answering in ontologies

An ontology imposes constraints on the data and actual data may be **incomplete** w.r.t. such constraints (**incomplete information assumption** on the data, as in **information integration** in databases).

The system has to **take into account** the constraints during query answering, and **overcome incompleteness**.



Query answering amounts to **logical inference**, which is computationally more costly than (classic) query answering over a database.

**Note:** the size of the data is not considered critical (comparable to the size of the intensional information) and queries are typically simple, i.e., atomic (a class name), and query answering amounts to instance checking.

# Query answering in ontologies – Example 1 (1\2)

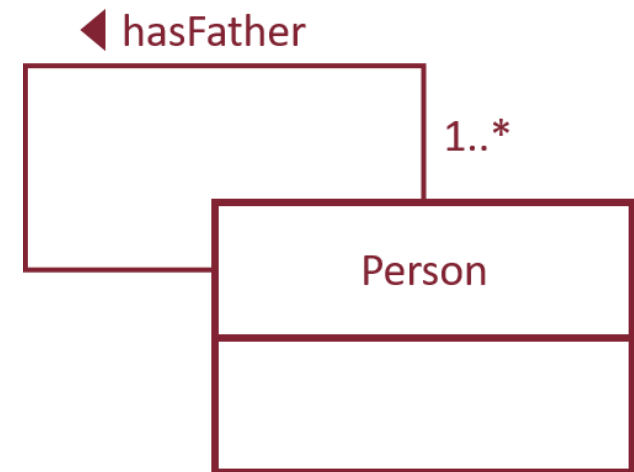
Consider an ontology modelling the paternity relationships between persons:  
"each Person has a father who is a Person"

The following TBox captures our domain:

**TBox T** = { **Person**  $\sqsubseteq$   $\exists$ hasFather,  
           $\exists$ hasFather  $\sqsubseteq$  **Person**  
           $\exists$ hasFather<sup>-</sup>  $\sqsubseteq$  **Person** }

Suppose our ABox contains the following facts:

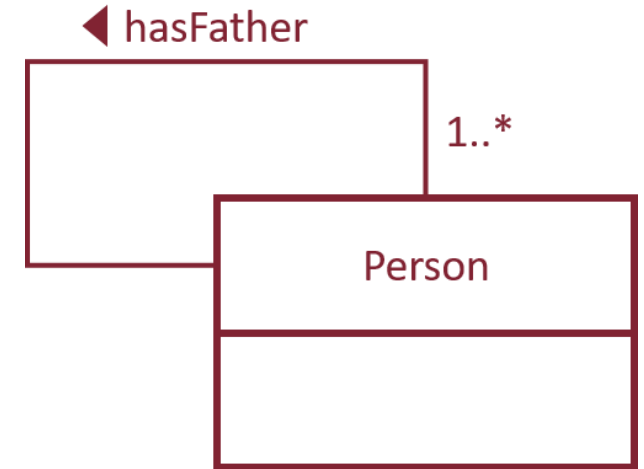
**ABox A** = { **Person(john)**, **Person(bob)**, **Person(tom)**,  
          **hasFather(john,bob)**, **hasFather(bob,tom)** }



# Query answering in ontologies – Example 1 (2\2)

$T = \{ \text{Person} \sqsubseteq \exists \text{hasFather},$   
 $\exists \text{hasFather} \sqsubseteq \text{Person},$   
 $\exists \text{hasFather}^- \sqsubseteq \text{Person} \}$

$A = \{ \text{Person}(\text{john}), \text{Person}(\text{bob}), \text{Person}(\text{tom}),$   
 $\text{hasFather}(\text{john}, \text{bob}), \text{hasFather}(\text{bob}, \text{tom}) \}$



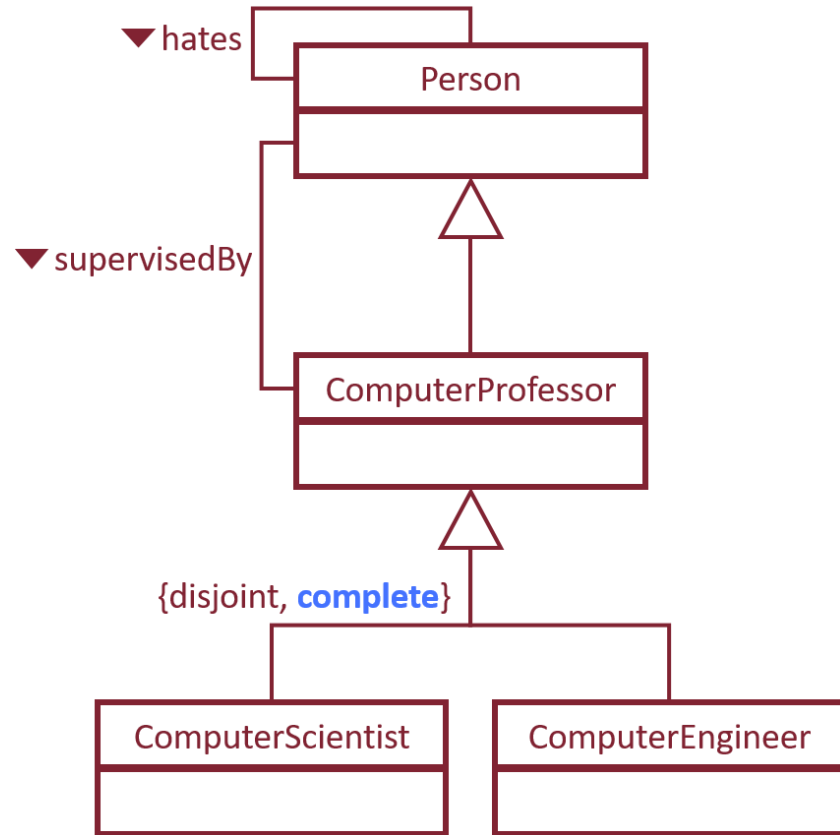
Queries:

- $q_1: \{(x, y) \mid \text{hasFather}(x, y)\}$
- $q_2: \{(x) \mid \exists y. \text{hasFather}(x, y)\}$
- $q_3: \{(x) \mid \exists y, v, w. \text{hasFather}(x, y) \wedge \text{hasFather}(y, v) \wedge \text{hasFather}(v, w)\}$
- $q_4: \{(x, w) \mid \exists y, v. \text{hasFather}(x, y) \wedge \text{hasFather}(y, v) \wedge \text{hasFather}(v, w)\}$

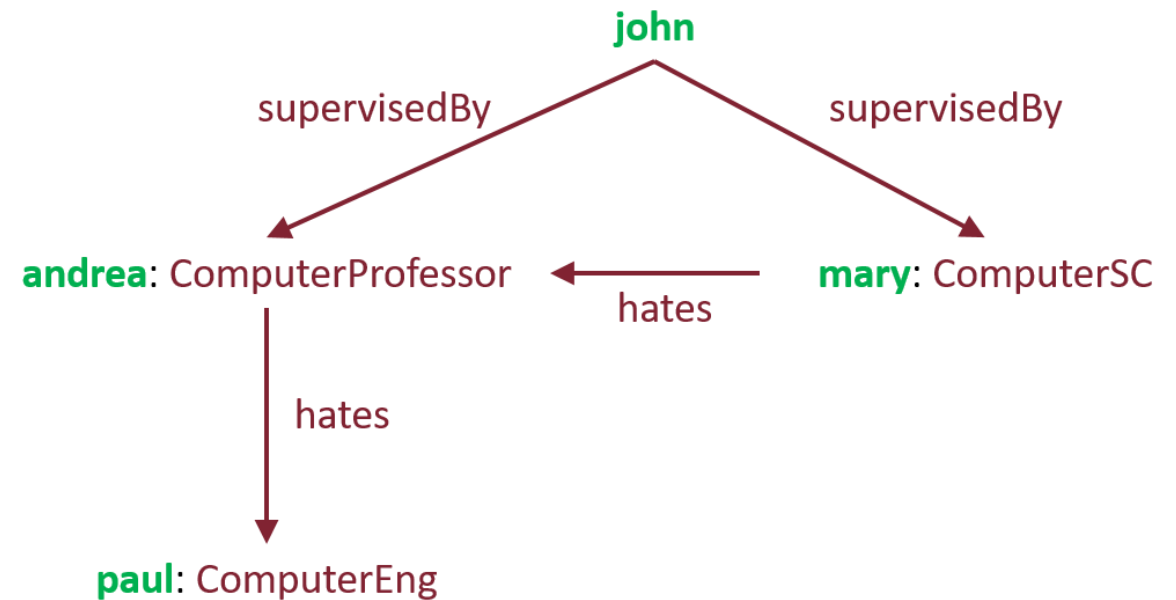
Answers:

- to  $q_1$ :  $\{(\text{john}, \text{bob}), (\text{bob}, \text{tom})\}$
- to  $q_2$ :  $\{\text{john}, \text{bob}, \text{tom}\}$
- to  $q_3$ :  $\{\text{john}, \text{bob}, \text{tom}\}$
- to  $q_4$ :  $\{\}$

# Query answering in ontologies – Example 2 (2\2)



Note that **ComputerProfessor** is partitioned into **ComputerScientist** and **ComputerEngineer**.

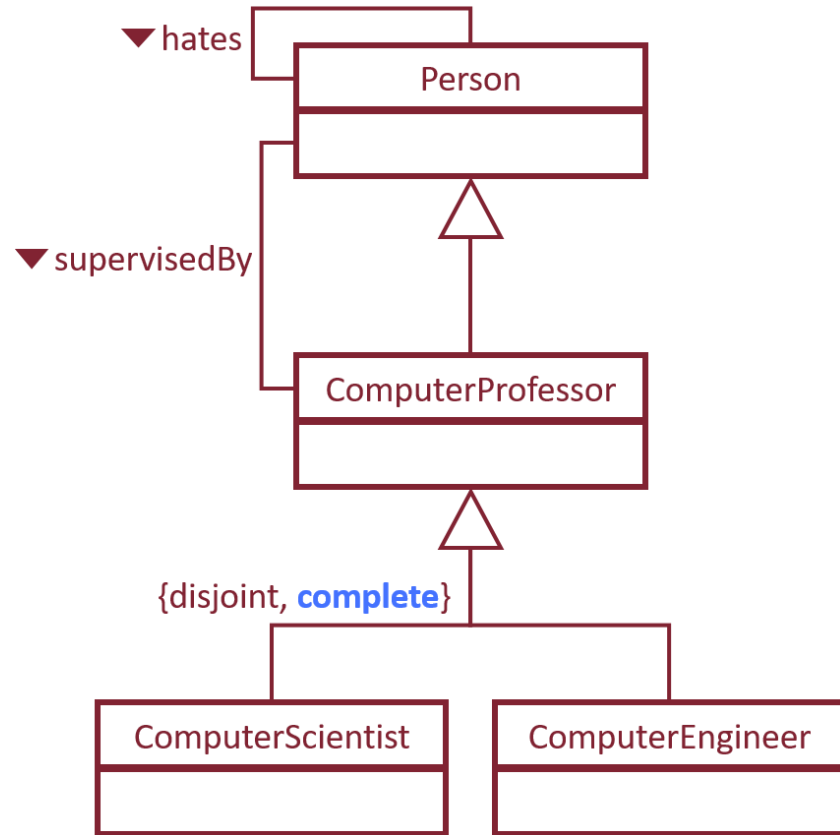


**Query:**  $\{ (x) \mid \exists y, z. \text{supervisedBy}(x, y) \wedge \text{ComputerSC}(y) \wedge \text{hates}(y, z) \wedge \text{ComputerEng}(z) \}$

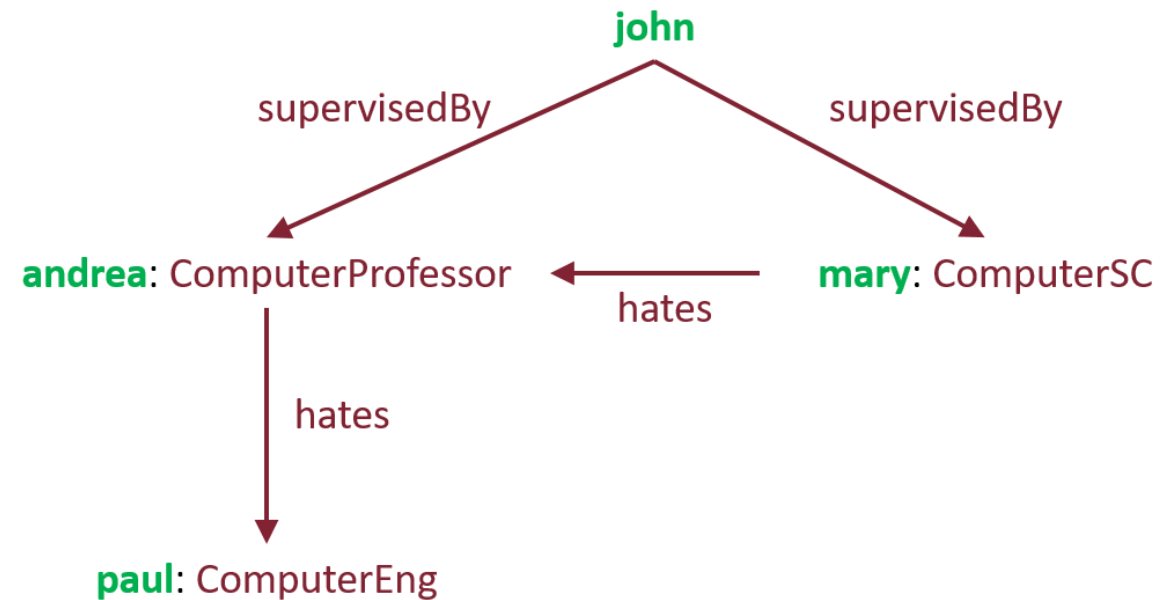
**Answer:** ???



# Query answering in ontologies – Example 2 (2\2)



Note that **ComputerProfessor** is partitioned into **ComputerScientist** and **ComputerEngineer**.



**Query:**  $\{ (x) \mid \exists y, z. \text{supervisedBy}(x, y) \wedge \text{ComputerSC}(y) \wedge \text{hates}(y, z) \wedge \text{ComputerEng}(z) \}$

**Answer:**  $\{ \text{john} \}$  → To get this answer we need to resort to **reasoning by cases**.

# Query answering in ontologies

Two borderline cases:

- Just **classes and properties** of the ontology → **instance checking**
  - Ontology languages are tailored for capturing intensional relationships.
  - They are **quite poor as query languages**: we cannot refer to same object via multiple navigation paths in the ontology, i.e., allow only for a limited form of join, namely chaining.
- Full **SQL** (or equivalently, **first-order logic**)
  - Problem: in the presence of **incomplete information**, query answering becomes **undecidable** (FOL validity).
- A good tradeoff is to use (unions of) **conjunctive queries**.

## Conjunctive Query (recap)

Let  $\varphi(x_1, \dots, x_n)$  be a FOL formula with free variables  $x_1, \dots, x_n$

$\varphi(x_1, \dots, x_n)$  is an **Existential Conjunction** if it is of the form

$$\exists y_1, \dots, \exists y_n P_1(\bar{z}_1) \wedge \dots \wedge P_k(\bar{z}_k)$$

A FOL query  $\{(x_1, \dots, x_n) | \varphi(x_1, \dots, x_n)\}$  is a **Conjunctive Query (CQ)** if  $\varphi(x_1, \dots, x_n)$  is an Existential Conjunction.

**NOTE:** In a **CQ** over a DL ontology the predicates  $P_i$  in the Existential Conjunction are only **atomic concepts** (unary predicates) and **atomic roles** (binary predicates)

# Union of Conjunctive Queries (recap)

- A formula  $\varphi(x_1, \dots, x_n)$  is Union of Existential Conjunctions if it is of the form

$$\bigvee_i \psi_i(x_1, \dots, x_n)$$

where each  $\psi_i$  is an existential conjunction.

**Observe:** same free variables.

- A FOL query  $\{(x_1, \dots, x_n) \mid \varphi(x_1, \dots, x_n)\}$  is a **Union of Conjunctive Queries** if  $\varphi(x_1, \dots, x_n)$  is a Union of Existential Conjunctions.

## Certain answers to a query

Let  $\mathbf{O} = \langle \mathbf{T}, \mathbf{A} \rangle$  be an **ontology**,  $I$  an **interpretation** for  $\mathbf{O}$ , and  $\mathbf{q} = \{\bar{x} \mid \exists \bar{y}. \varphi(\bar{x}, \bar{y})\}$  be a **CQ**.

- **Definition:** The answer to  $\mathbf{q}$  over  $I$ , denoted  $q^I$ , is the set of **tuples  $\bar{c}$  of constants of  $\mathbf{A}$**  such that the formula  $\exists \bar{y}. \varphi(\bar{c}, \bar{y})$  evaluates to true in  $I$ .

We are interested in finding those answers that hold in **all models of an ontology**.

- **Definition:** The **certain answers** to  $\mathbf{q}$  over  $\mathbf{O} = \langle \mathbf{T}, \mathbf{A} \rangle$ , denoted **cert( $\mathbf{q}, \mathbf{O}$ )** are the tuples  $\bar{c}$  of constants of  $\mathbf{A}$  such that  $\bar{c} \in q^I$ , **for every model  $I$  of  $\mathbf{O}$ , i.e.,  $\forall I \in \text{Mod}(\mathbf{O})$**

# Query answering in ontologies

- **Definition:** **Query answering** over an ontology  $\mathcal{O}$  is the problem of computing the **certain answer** to a query over  $\mathcal{O}$

Computing certain answers is a form of **logical implication**:

$$\bar{c} \in \text{cert}(\{\bar{x} \mid \exists \bar{y}. \varphi(\bar{x}, \bar{y})\}, \mathcal{O}) \quad \text{iff} \quad \mathcal{O} \models \exists \bar{y}. \varphi(\bar{c}, \bar{y})$$

**Note:** A special case of query answering is **instance checking**: it amounts to answering the Boolean query  $q = \{() \mid A(c)\}$  (resp.,  $q = \{() \mid P(c, c')\}$ ) over  $\mathcal{O}$ .

**Remember:** in case of Boolean queries the answer is the **empty set** (negative answer) or the **empty tuple** (positive answer).

# Query answering in ontologies – Example 1 (2\2)

$T = \{ \text{Person} \sqsubseteq \exists \text{hasFather},$   
 $\exists \text{hasFather} \sqsubseteq \text{Person},$   
 $\exists \text{hasFather}^- \sqsubseteq \text{Person} \}$

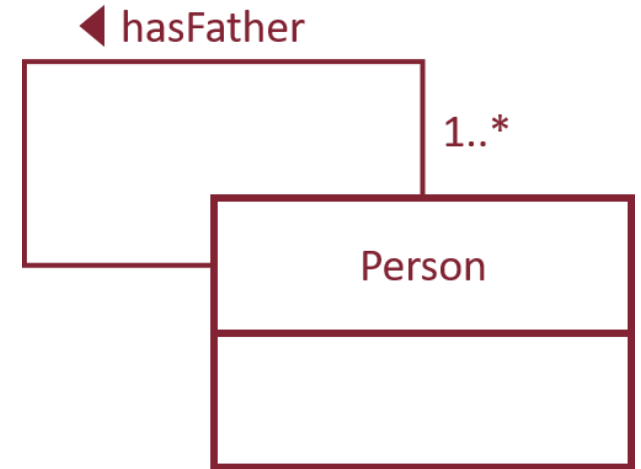
$A = \{ \text{Person}(\text{john}), \text{Person}(\text{bob}), \text{Person}(\text{tom}),$   
 $\text{hasFather}(\text{john}, \text{bob}), \text{hasFather}(\text{bob}, \text{tom}) \}$

Queries:

- $q_1: \{(x, y) \mid \text{hasFather}(x, y)\}$
- $q_2: \{(x) \mid \exists y. \text{hasFather}(x, y)\}$
- $q_3: \{(x) \mid \exists y, v, w. \text{hasFather}(x, y) \wedge \text{hasFather}(y, v) \wedge \text{hasFather}(v, w)\}$
- $q_4: \{(x, w) \mid \exists y, v. \text{hasFather}(x, y) \wedge \text{hasFather}(y, v) \wedge \text{hasFather}(v, w)\}$

**Certain answers:**

- $\text{cert}(q_1, O): \{(\text{john}, \text{bob}), (\text{bob}, \text{tom})\}$
- $\text{cert}(q_2, O): \{\text{john}, \text{bob}, \text{tom}\}$
- $\text{cert}(q_3, O): \{\text{john}, \text{bob}, \text{tom}\}$
- $\text{cert}(q_4, O): \{\}$



# Complexity measures for queries over ontologies

When measuring the complexity of answering a query  $q$  over an ontology  $O = \langle T, A \rangle$ , various parameters are of importance.

Depending on which parameters we consider, we get different complexity measures:

- **Data complexity**: only the size of the **ABox** (i.e., the data) matters. **TBox** and **query** are considered fixed.
- **Query complexity**: only the size of the query matters. **TBox** and **ABox** are considered fixed.
- **Schema complexity**: only the size of the **TBox** (i.e., the schema) matters. **ABox** and **query** are considered fixed.
- **Combined complexity**: no parameter is considered fixed.

**Note:** In the **OBDM setting**, the size of the **data** largely dominates the size of the conceptual layer (and of the query) → **Data complexity is the relevant complexity measure.**



# Recognition problem for query answering

When studying the complexity of **query answering**, we need to consider the associated **decision problem**:

**Definition [Recognition problem for query answering over an ontology]:**

Given an ontology  $\mathbf{O} = \langle \mathbf{T}, \mathbf{A} \rangle$ , a query  $\mathbf{q}$  over  $\mathbf{O}$ , and a tuple  $\bar{c}$  of constants, check whether  $\bar{c} \in \mathbf{cert}(\mathbf{q}, \mathbf{O})$ .

We look mainly at the data complexity of query answering, i.e., complexity of the recognition problem computed w.r.t. the size of the **ABox**  $\mathbf{A}$  only.

# Complexity of query answering in DLs

Studied extensively for (unions of) CQs and various ontology languages:

|                            | Combined complexity      | Data complexity           |
|----------------------------|--------------------------|---------------------------|
| Plain databases            | <b>NP-complete</b>       | <b>AC<sup>0</sup> (1)</b> |
| <i>ALCI, SH, SHIQ, ...</i> | <b>2ExpTime-complete</b> | <b>coNP-complete (2)</b>  |
| OWL 2                      | <b>3ExpTime-hard</b>     | <b>coNP-hard</b>          |

**(1) This is what we need to scale with the data.**

**(2) coNP-hard already for a TBox with a single disjunction**

## Questions:

- Can we find interesting description logics for which query answering can be done efficiently (i.e., in **AC<sup>0</sup>**)?
- If yes, can we leverage **relational database technology** for query answering?  
(We need this for OBDM)

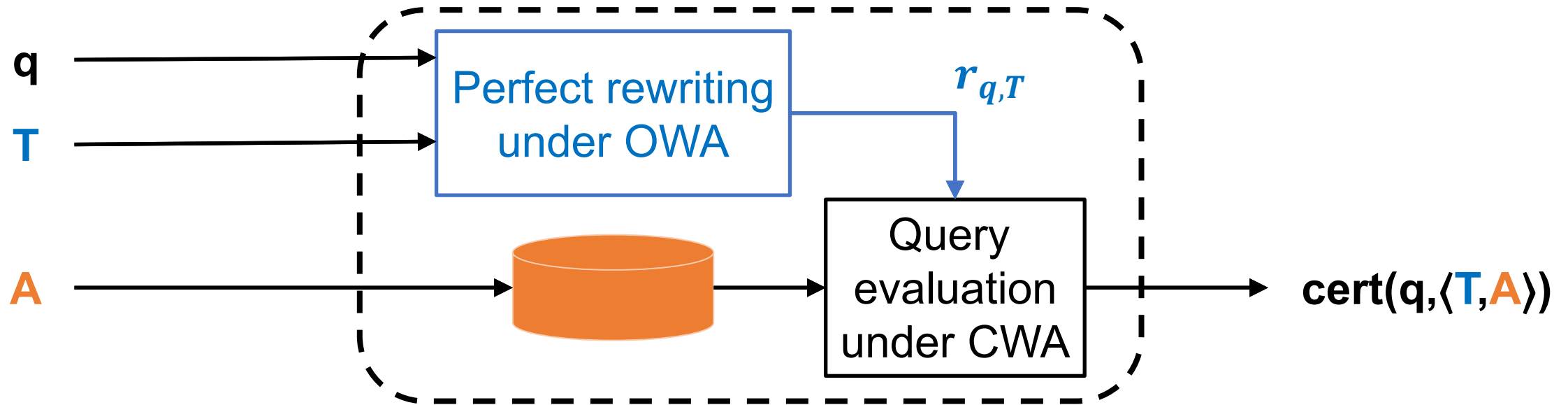
## Inference in query answering



To be able to deal with data efficiently, we need to separate the contribution of  $A$  from the contribution of  $q$  and  $T$

→ Query answering by **query rewriting**.

# Query answering by query rewriting



Query answering can be thought as done in two phases:

- **Perfect rewriting**: produce from  $q$  and the TBox  $T$  a new query  $r_{q,T}$  (called the **perfect rewriting** of  $q$  w.r.t.  $T$  ).
- **Query evaluation**: evaluate  $r_{q,T}$  over the ABox  $A$  seen as a complete database (without considering the TBox  $T$  ). The result of such evaluation is  $\text{cert}(q, \langle T, A \rangle)$

# $Q$ -rewritability

Let  $Q$  be a query language and  $\mathcal{L}$  an ontology language.

- **Definition [ $Q$ -rewritability]:** For an ontology language  $\mathcal{L}$ , query answering is  $Q$ -rewritable if for every TBox  $T$  of  $\mathcal{L}$  and for every query  $q$ , the perfect reformulation  $r_{q,T}$  of  $q$  w.r.t.  $T$  can be expressed in the query language  $Q$ .

Notice that the complexity of computing  $r_{q,T}$  or the size of  $r_{q,T}$  do **not** affect data complexity.

Hence,  $Q$ -rewritability is tightly related to **data complexity**, i.e.:

- the complexity of computing  $\text{cert}(q, \langle T, A \rangle)$  measured in the size of the ABox  $A$  only,
- which corresponds to the **complexity of evaluating  $r_{q,T}$  over  $A$  (seeing  $A$  as a database)**

# $Q$ -rewritability and OBDM

The **expressiveness of the ontology language** affects the rewriting language, i.e., the language into which we are able to rewrite UCQs:

We are especially interested in **FOL-rewritability**:

- The rewriting  $r_{q,T}$  can be expressed in FOL, i.e., in **SQL**.
- Recall evaluating FOL\SQL queries over a **plain database** is in **AC<sup>0</sup>** in data complexity.
- Query evaluation can be **delegated to a relational DBMS**.

## Credits

The presentations of this module of the course are based on the original slides of Prof. Giuseppe De Giacomo, Prof. Diego Calvanese, Prof. Marco Console, Prof. Maurizio Lenzerini, and Prof. Domenico Lembo.

---